Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Ca' Foscari
University
of Venice

Model Context Protocol
MCP

Ca' Foscari University of Venice

The AI "Context Chaos"

- Large Language Models (LLMs) are powerful but "stateless" (like "brains in a jar"). They have no access to the outside world.
- To be useful, an application must provide Context (e.g., user files, database schemas, API definitions, terminal output).
- The Methodological Problem: before MCP, every AI agent, framework, and application developer invented their own proprietary, non-standard way to handle this context.
- This creates classic software engineering failures:
  - Tight Coupling: The application is "stuck" to one specific AI model, agent, or vendor (e.g., the OpenAI Assistants API).
  - Development Silos: Tools built for one agent framework cannot be used with another.
  - Poor Maintainability: Changing the AI backend (e.g., from GPT-4 to Claude 3) requires a major rewrite.

Ca' Foscari University of Venice

Model Context Protocol (MCP)

- MCP is a standardized specification and design pattern for AI-application communication.
- It is an application-layer standard for interoperability.
- MCP defines a universal, "pluggable" interface for an application to provide Context and Tools to any AI model or agent.
- Think of MCP as the "USB-C Port for AI Agents.
  - It doesn't matter who makes the "laptop" (the application) or the "device" (the AI model); if they both use the standard, they just work together.

How MCP Works: A Simplified View

Client (the application)

- This is your IDE (e.g., VS Code), your web app, or your custom software.
- It provides the context: "Here are the files in the user's project," or "Here is the database schema."
- It offers tools: "You have a tool called `run_terminal_command` and another called `search_database`."

Server (AI Agent/LLM)

- This is the "brain" (e.g., GPT-4, Llama 3).
- It receives the context.
- It requests to use a tool: "Please execute `run_terminal_command` with the argument `pytest`."

The protocol standardizes this request/response loop, abstracting away the specifics of which model is being used.

Why This Matters

- MCP is a fundamental methodological shift in AI engineering.
- Promotes True Decoupling:
  - The application (frontend/logic) is fully decoupled from the AI model (the "brain").
  - This is a core tenet of good software design: Separation of Concerns.
- Enables Interoperability
  - An enterprise can build *one* application that can use a public cloud model (like GPT-4) or a private, fine-tuned local model (like Llama 3) just by changing a config file.
- Prevents Vendor Lock-In
- Fosters a Composable Ecosystem

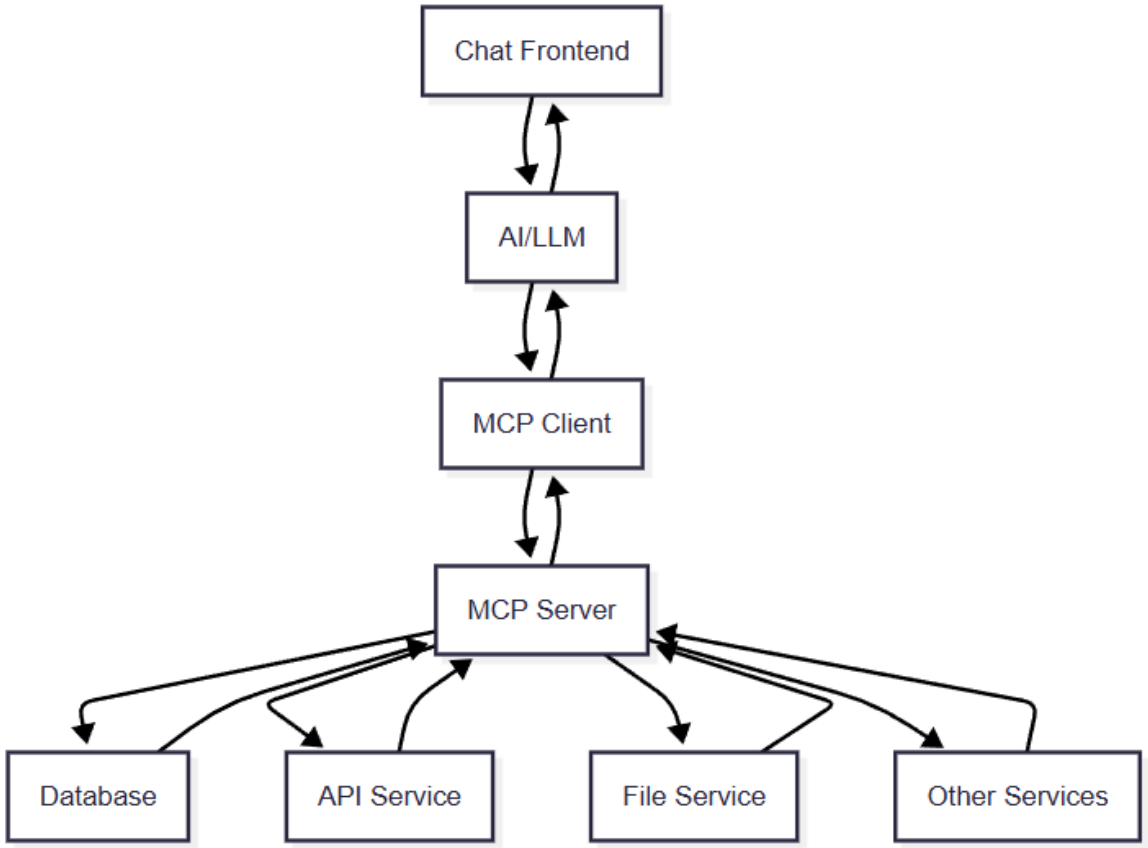
Applications & Use Cases

- AI Code Assistants (Co-pilots):
  - This is the primary driver.
  - Projects like Continue (open-source-copilot) use MCP to integrate multiple LLMs (local and cloud-based) directly into IDEs like VS Code and JetBrains.
  - The IDE (Client) provides the code context via MCP to any compatible LLM (Server).
- Complex, Multi-Step AI Agents:
  - Standardizes how agents (like Auto-GPT) interact with external "tools" (APIs, filesystems, databases) to complete complex tasks.
- Advanced RAG (Retrieval-Augmented Generation):
  - MCP provides a standard way for the Retriever (which finds data) to "feed" that data as context to the Generator (the LLM).

Conclusion

- The first wave of AI development was chaotic, leading to tightly-coupled, monolithic systems.
- MCP provides the methodology to fix this.
- It moves AI development from a "Wild West" phase into a mature engineering discipline based on proven principles:
  - Decoupling
  - Interoperability
  - Abstraction
- It gives developers flexibility, prevents vendor lock-in, and ensures future-proof applications.

Diagram MCP


Chat Frontend
AI/LLM
MCP Client
MCP Server
Database
API Service
File Service
Other Services